

racing_rl

March 23, 2026

1 Racing Multi-Agente con DQN

1.1 Objetivos de este Notebook

En este notebook aprenderás:

1. **Arquitectura multi-agente:** ¿Los coches comparten red neuronal o son independientes?
 2. **Diseño del estado:** Cómo funcionan los sensores raycast
 3. **Espacio de acciones:** Por qué 9 acciones discretas
 4. **Función de recompensa:** Cómo incentivar el comportamiento deseado
 5. **Variantes de arquitectura:** Implementaciones alternativas
-

1.2 Prerequisitos

- Conceptos básicos de DQN (Experience Replay, Target Network)
- PyTorch básico
- Pygame (para visualización)

```
[1]: # Imports necesarios
import sys
import os
from pathlib import Path

# Añadir el directorio actual al path para importar racing_game
RACING_DIR = Path().absolute()
if str(RACING_DIR) not in sys.path:
    sys.path.insert(0, str(RACING_DIR))

import numpy as np
import matplotlib.pyplot as plt
from collections import deque
import random
import math

# PyTorch
import torch
import torch.nn as nn
import torch.optim as optim
```

```
# Pygame (para visualización)
import pygame

print(f"PyTorch: {torch.__version__}")
print(f"CUDA disponible: {torch.cuda.is_available()}")
print(f"Directorio: {RACING_DIR}")
```

```
pygame 2.6.1 (SDL 2.32.64, Python 3.14.3)
Hello from the pygame community. https://www.pygame.org/contribute.html
PyTorch: 2.10.0+cu128
CUDA disponible: False
Directorio: /home/jeironpro/OPT-ML/Reinforcement
Learning/03_proyectos_dqn/racing

<frozen importlib._bootstrap>:491: RuntimeWarning: Your system is avx2 capable
but pygame was not built with support for it. The performance of some of your
blits could be adversely affected. Consider enabling compile time detection with
environment variables like PYGAME_DETECT_AVX2=1 if you are compiling without
cross compilation.
/home/jeironpro/.local/lib/python3.14/site-packages/pygame/pkgdata.py:25:
UserWarning: pkg_resources is deprecated as an API. See
https://setuptools.pypa.io/en/latest/pkg_resources.html. The pkg_resources
package is slated for removal as early as 2025-11-30. Refrain from using this
package or pin to Setuptools<81.
    from pkg_resources import resource_stream, resource_exists
```

2 1. Descripción del Juego

2.1 ¿Qué es Racing Multi-Agente?

Un juego donde 4 **coches** (configurable) compiten simultáneamente en un circuito ovalado. Cada coche:

- Tiene **sensores** que detectan la distancia a los bordes
- Aprende a **acelerar, frenar y girar** para mantenerse en la pista
- **Muere** si choca con los bordes

CIRCUITO OVALADO

← Coches compitiendo

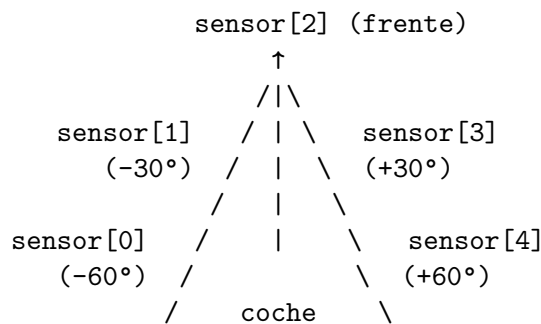
3 2. Análisis de la Arquitectura

3.1 2.1 Estado / Observación (6 dimensiones)

Cada coche recibe un **vector de 6 valores**:

Índice	Componente	Rango	Descripción
0	sensor_izq_60°	[0, 1]	Distancia al borde a -60°
1	sensor_izq_30°	[0, 1]	Distancia al borde a -30°
2	sensor_frente	[0, 1]	Distancia al borde al frente
3	sensor_der_30°	[0, 1]	Distancia al borde a +30°
4	sensor_der_60°	[0, 1]	Distancia al borde a +60°
5	velocidad	[0, 1]	Velocidad normalizada

3.1.1 Diagrama de Sensores Raycast



Valor del sensor: - 1.0 = Lejos del borde (seguro) - 0.0 = Muy cerca del borde (peligro)

```
[2]: # Visualizar cómo funciona el estado
# Importamos la clase Car del juego original
from racing_game import Car, Track, WINDOW_WIDTH, WINDOW_HEIGHT

# Crear una pista y un coche de ejemplo
track = Track(WINDOW_WIDTH, WINDOW_HEIGHT)
start_positions = track.get_start_positions(1)
pos, angle = start_positions[0]

car = Car(0, (255, 0, 0), pos, angle)

# Obtener estado inicial
car._update_sensors(track)
state = car.get_state()

print("="*60)
print("ESTADO DEL COCHE (6 dimensiones)")
print("="*60)
```

```

print(f"\nVector de estado: {state}")
print(f"\nDesglose:")
sensor_names = ["Izq 60°", "Izq 30°", "Frente", "Der 30°", "Der 60°",
↪ "Velocidad"]
for i, (name, value) in enumerate(zip(sensor_names, state)):
    bar = " " * int(value * 20) + " " * (20 - int(value * 20))
    print(f" [{i}] {name:12s}: {value:.3f} |{bar}|")

```

=====

ESTADO DEL COCHE (6 dimensiones)

=====

Vector de estado: [0.37333333 0.54 0.97333336 0.97333336 0.64 0.]

Desglose:

[0]	Izq 60°	: 0.373		
[1]	Izq 30°	: 0.540		
[2]	Frente	: 0.973		
[3]	Der 30°	: 0.973		
[4]	Der 60°	: 0.640		
[5]	Velocidad	: 0.000		

3.2 2.2 Espacio de Acciones (9 acciones discretas)

El coche puede realizar **9 acciones diferentes**, combinando aceleración y dirección:

Acción	Aceleración	Dirección	Descripción
0	-	-	Nada (inercia)
1	Acelerar	-	Solo acelerar
2	Frenar	-	Solo frenar
3	-	← Izquierda	Solo girar izquierda
4	-	→ Derecha	Solo girar derecha
5	Acelerar	← Izquierda	Acelerar + Izquierda
6	Acelerar	→ Derecha	Acelerar + Derecha
7	Frenar	← Izquierda	Frenar + Izquierda
8	Frenar	→ Derecha	Frenar + Derecha

3.2.1 Física del Giro

Importante: El giro solo funciona si el coche se está moviendo.

```

if self.speed > 0.1: # Necesita velocidad mínima para girar
    turn_factor = min(1.0, self.speed / 3.0) # Giro más efectivo a velocidad media
    actual_turn = self.turn_speed * turn_factor

```

Esto simula **física realista**: no puedes girar el volante si estás parado.

```
[3]: # Demostrar el efecto de cada acción
print("="*60)
print("ESPACIO DE ACCIONES")
print("="*60)

acciones = [
    (0, "Nada (inercia)", "-", "-"),
    (1, "Acelerar", "↑", "-"),
    (2, "Frenar", "↓", "-"),
    (3, "Girar izquierda", "-", "←"),
    (4, "Girar derecha", "-", "→"),
    (5, "Acelerar + Izq", "↑", "←"),
    (6, "Acelerar + Der", "↑", "→"),
    (7, "Frenar + Izq", "↓", "←"),
    (8, "Frenar + Der", "↓", "→"),
]

print(f"\n{'Acción':<8} {'Descripción':<20} {'Acel':<6} {'Dir':<6}")
print("-" * 45)
for a, desc, acel, dir in acciones:
    print(f"{a:<8} {desc:<20} {acel:<6} {dir:<6}")
```

```
=====
ESPACIO DE ACCIONES
=====
```

Acción	Descripción	Acel	Dir

0	Nada (inercia)	-	-
1	Acelerar	↑	-
2	Frenar	↓	-
3	Girar izquierda	-	←
4	Girar derecha	-	→
5	Acelerar + Izq	↑	←
6	Acelerar + Der	↑	→
7	Frenar + Izq	↓	←
8	Frenar + Der	↓	→

3.3 2.3 Función de Recompensa

La recompensa incentiva al coche a **avanzar** y **evitar colisiones**:

Evento	Recompensa	Propósito
Avanzar	$+0.1 \times \Delta \text{distancia}$	Incentivar movimiento
Moverse ($v > 0$)	+0.1	Evitar quedarse parado
Colisión	-10.0	Penalizar choques

3.3.1 Código de la recompensa (racing_game.py:372-382)

```
if not car.alive:
    reward = -10 # Penalización por colisión
else:
    reward = (car.distance - old_distance) * 0.1 # Recompensa por avanzar
    if car.speed > 0:
        reward += 0.1 # Bonus por moverse
```

3.3.2 ¿Por qué este diseño?

1. **Recompensa densa:** El agente recibe feedback continuo, no solo al final
2. **Bonus por velocidad:** Evita que el agente aprenda a quedarse quieto
3. **Penalización fuerte:** -10 es significativo comparado con +0.1 por paso

3.4 2.4 Arquitectura del Agente

3.4.1 PREGUNTA CLAVE: ¿Los 4 coches comparten la misma red neuronal?

3.5 NO. Cada coche tiene su propio agente completamente independiente.

En racing_game.py:544-554:

```
agents = [CarAgent(i) for i in range(n_cars)] # 4 agentes separados
```

3.5.1 Cada CarAgent contiene:

Componente	Descripción	Parámetros
q_network	Red DQN principal	18,569
target_network	Copia para estabilidad	18,569
optimizer	Adam (lr=0.001)	-
memory	Replay buffer	50,000 experiencias

3.5.2 Arquitectura de la Red (CarDQN)

Estado [6] → Linear(128) → ReLU → Linear(128) → ReLU → Q-valores [9]

Parámetros:

Capa 1: $6 \times 128 + 128 = 896$
Capa 2: $128 \times 128 + 128 = 16,512$
Capa 3: $128 \times 9 + 9 = 1,161$
TOTAL: 18,569 parámetros por agente

Con 4 coches: $18,569 \times 4 = 74,276$ parámetros totales

```
[4]: # Verificar la arquitectura original
from racing_game import CarDQN, CarAgent

# Crear red y contar parámetros
red = CarDQN(state_size=6, n_actions=9)
```

```

n_params = sum(p.numel() for p in red.parameters())

print("="*60)
print("ARQUITECTURA CarDQN (Original)")
print("="*60)
print(f"\n{red}")
print(f"\nParámetros totales: {n_params:,}")
print(f"Con 4 coches independientes: {n_params * 4:,} parámetros")

```

```

=====
ARQUITECTURA CarDQN (Original)
=====

```

```

CarDQN(
  (net): Sequential(
    (0): Linear(in_features=6, out_features=128, bias=True)
    (1): ReLU()
    (2): Linear(in_features=128, out_features=128, bias=True)
    (3): ReLU()
    (4): Linear(in_features=128, out_features=9, bias=True)
  )
)

```

Parámetros totales: 18,569

Con 4 coches independientes: 74,276 parámetros

3.5.3 ¿Por qué agentes independientes?

Enfoque	Ventajas	Desventajas
Independiente (actual)	Simple, paralelo, sin interferencia	Más parámetros, sin transfer
Red compartida	Menos parámetros, transfer learning	Competencia por representación
Evolutivo	Mejor exploración global	Convergencia más lenta

En las siguientes secciones implementaremos **todas las variantes**.

4 3. Código Original Explicado

Importamos las clases principales del archivo `racing_game.py`:

```

[5]: # Importar todo el módulo
from racing_game import (
    Car, Track, RacingGame,
    CarDQN, CarAgent,

```

```

WINDOW_WIDTH, WINDOW_HEIGHT,
CAR_COLORS
)

print("Clases importadas:")
print(" - Car: Representa un coche individual")
print(" - Track: El circuito")
print(" - RacingGame: Gestiona el juego completo")
print(" - CarDQN: Red neuronal DQN")
print(" - CarAgent: Agente completo con entrenamiento")

```

Clases importadas:

- Car: Representa un coche individual
 - Track: El circuito
 - RacingGame: Gestiona el juego completo
 - CarDQN: Red neuronal DQN
 - CarAgent: Agente completo con entrenamiento
-

5 4. Entrenamiento Rápido (Demo)

Entrenamos por pocos episodios para ver el proceso:

```

[6]: def entrenar_demo(n_cars=4, episodes=20, max_steps=500, render_cada=5):
    """
    Entrena agentes DQN independientes (arquitectura original).

    Args:
        n_cars: Número de coches
        episodes: Episodios de entrenamiento
        max_steps: Pasos máximos por episodio
        render_cada: Renderizar cada N episodios (0 = nunca)
    """
    print(f"\n{'='*60}")
    print(f"ENTRENAMIENTO DEMO - {n_cars} Coches Independientes")
    print(f"{'='*60}")

    # Crear juego (con renderizado si queremos visualizar)
    render = render_cada > 0
    game = RacingGame(n_cars=n_cars, render=render)

    # Crear agentes independientes
    agents = [CarAgent(i) for i in range(n_cars)]

    # Métricas
    fitness_history = [[] for _ in range(n_cars)]

```

```

for ep in range(episodes):
    states = game.reset()

    for step in range(max_steps):
        # Cada agente decide su acción
        actions = [agent.act(state) for agent, state in zip(agents, states)]

        # Ejecutar paso
        next_states, rewards, dones, all_done = game.step(actions)

        # Almacenar experiencias y entrenar
        for i, agent in enumerate(agents):
            agent.remember(states[i], actions[i], rewards[i],
↪next_states[i], dones[i])
            agent.replay()

        states = next_states

        # Renderizar
        if render and (ep % render_cada == 0):
            game.render()
            game.clock.tick(60)

        # Procesar eventos pygame
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                game.close()
                return agents, fitness_history

        if all_done:
            break

    # Actualizar agentes
    for i, agent in enumerate(agents):
        agent.decay_epsilon()
        if ep % 10 == 0:
            agent.update_target()
            fitness_history[i].append(game.cars[i].fitness)

    # Log
    if (ep + 1) % 5 == 0:
        avg_fitness = np.mean([car.fitness for car in game.cars])
        print(f"Ep {ep+1:3d} | Fitness: {avg_fitness:.0f} | : {agents[0].
↪epsilon:.3f}")

    if render:
        game.close()

```

```

    return agents, fitness_history

# Entrenar demo (sin visualización para ir más rápido)
agents_original, history_original = entrenar_demo(n_cars=4, episodes=30,
↪render_cada=0)

```

ENTRENAMIENTO DEMO - 4 Coches Independientes

```

=====
Ep   5 | Fitness: 208 | : 0.975
Ep  10 | Fitness: 171 | : 0.951
Ep  15 | Fitness: 221 | : 0.928
Ep  20 | Fitness: 209 | : 0.905
Ep  25 | Fitness: 207 | : 0.882
Ep  30 | Fitness: 180 | : 0.860

```

```

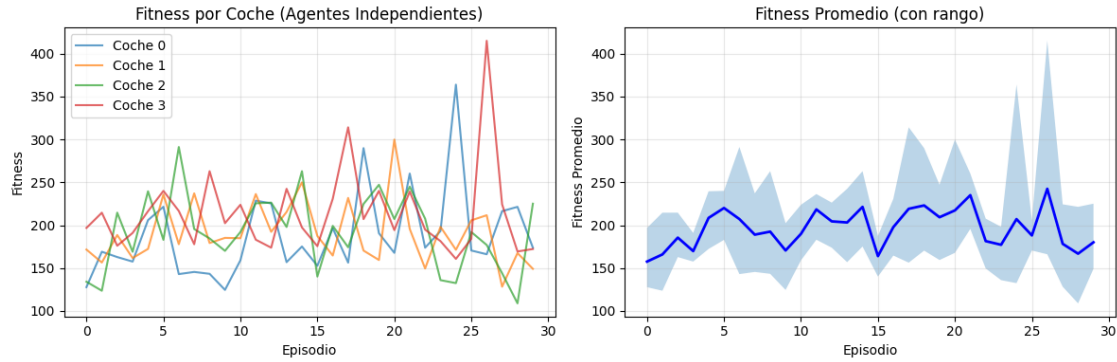
[7]: # Visualizar curvas de aprendizaje
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
for i, hist in enumerate(history_original):
    plt.plot(hist, label=f'Coche {i}', alpha=0.7)
plt.xlabel('Episodio')
plt.ylabel('Fitness')
plt.title('Fitness por Coche (Agentes Independientes)')
plt.legend()
plt.grid(True, alpha=0.3)

plt.subplot(1, 2, 2)
avg_fitness = np.mean(history_original, axis=0)
plt.plot(avg_fitness, 'b-', linewidth=2)
plt.fill_between(range(len(avg_fitness)),
                 np.min(history_original, axis=0),
                 np.max(history_original, axis=0),
                 alpha=0.3)
plt.xlabel('Episodio')
plt.ylabel('Fitness Promedio')
plt.title('Fitness Promedio (con rango)')
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```



6 5. VARIANTES DE ARQUITECTURA

A continuación implementamos **5 variantes** de la arquitectura multi-agente:

Variante	Descripción	Diferencia clave
A	SharedDQN	Red compartida con cabezas independientes
B	CompetitiveDQN	Recompensa basada en ranking
C	CooperativeDQN	Recompensa por rendimiento grupal
D	A2C Multi-Agente	Policy Gradient en vez de DQN
E	EvolutionaryDQN	Selección de mejores agentes

6.1 Variante A: SharedDQN (Red Compartida)

6.1.1 Idea

En lugar de 4 redes independientes, usamos **una sola red** con: - **Encoder compartido**: Aprende representaciones comunes - **Cabezas independientes**: Una por coche para especialización

6.1.2 Ventajas

- Menos parámetros (18K vs 72K)
- Transfer de conocimiento entre coches
- Entrenamiento más eficiente

6.1.3 Desventajas

- Los coches compiten por la representación
- Puede ser menos flexible

```
[8]: class SharedDQN(nn.Module):
    """
    Red DQN con encoder compartido y cabezas independientes por coche.
```

Arquitectura:

Estado [6] → Shared[128] → ReLU → Shared[128] → ReLU

↓

↓ ↓ ↓ ↓
Head_0[9] Head_1[9] Head_2[9] Head_3[9]

"""

```
def __init__(self, state_size=6, n_actions=9, n_cars=4):
    super().__init__()
    self.n_cars = n_cars

    # Encoder compartido
    self.shared = nn.Sequential(
        nn.Linear(state_size, 128),
        nn.ReLU(),
        nn.Linear(128, 128),
        nn.ReLU()
    )

    # Cabezas independientes (una por coche)
    self.heads = nn.ModuleList([
        nn.Linear(128, n_actions) for _ in range(n_cars)
    ])

def forward(self, x, car_id=None):
    """
    Args:
        x: Estado (batch, 6) o (6,)
        car_id: ID del coche (0-3). Si None, devuelve para todos.
    """
    shared_features = self.shared(x)

    if car_id is not None:
        return self.heads[car_id](shared_features)
    else:
        # Devolver Q-valores para todos los coches
        return [head(shared_features) for head in self.heads]

class SharedCarAgent:
    """
    Agente multi-coche con red compartida.
    """

    def __init__(self, n_cars=4, state_size=6, n_actions=9):
        self.n_cars = n_cars
```

```

self.n_actions = n_actions
self.gamma = 0.95
self.epsilon = 1.0
self.epsilon_min = 0.05
self.epsilon_decay = 0.995

self.device = torch.device("cuda" if torch.cuda.is_available() else ↵
↵"cpu")

# UNA sola red para todos
self.q_network = SharedDQN(state_size, n_actions, n_cars).to(self.
↵device)
self.target_network = SharedDQN(state_size, n_actions, n_cars).to(self.
↵device)
self.target_network.load_state_dict(self.q_network.state_dict())

# UN solo optimizer
self.optimizer = optim.Adam(self.q_network.parameters(), lr=0.001)

# Memorias separadas por coche (para diversidad)
self.memories = [deque(maxlen=50000) for _ in range(n_cars)]
self.batch_size = 32

def act(self, state, car_id):
    if random.random() < self.epsilon:
        return random.randint(0, self.n_actions - 1)

    with torch.no_grad():
        state_t = torch.FloatTensor(state).unsqueeze(0).to(self.device)
        q_values = self.q_network(state_t, car_id)
        return q_values.argmax().item()

def remember(self, car_id, state, action, reward, next_state, done):
    self.memories[car_id].append((state, action, reward, next_state, done))

def replay(self):
    """Entrena con experiencias de TODOS los coches."""
    total_loss = 0

    for car_id in range(self.n_cars):
        if len(self.memories[car_id]) < self.batch_size:
            continue

        batch = random.sample(self.memories[car_id], self.batch_size)
        states, actions, rewards, next_states, dones = zip(*batch)

        states_t = torch.FloatTensor(np.array(states)).to(self.device)

```

```

        actions_t = torch.LongTensor(actions).to(self.device)
        rewards_t = torch.FloatTensor(rewards).to(self.device)
        next_states_t = torch.FloatTensor(np.array(next_states)).to(self.
↪device)

        dones_t = torch.FloatTensor(dones).to(self.device)

        # Q-valores para este coche específico
        q_values = self.q_network(states_t, car_id).gather(1, actions_t.
↪unsqueeze(1)).squeeze()

        with torch.no_grad():
            next_q = self.target_network(next_states_t, car_id).max(1)[0]
            target = rewards_t + (1 - dones_t) * self.gamma * next_q

        loss = nn.MSELoss()(q_values, target)
        total_loss += loss.item()

        self.optimizer.zero_grad()
        loss.backward()
        self.optimizer.step()

    return total_loss / self.n_cars

def update_target(self):
    self.target_network.load_state_dict(self.q_network.state_dict())

def decay_epsilon(self):
    self.epsilon = max(self.epsilon_min, self.epsilon * self.epsilon_decay)

# Verificar arquitectura
shared_net = SharedDQN(state_size=6, n_actions=9, n_cars=4)
n_params_shared = sum(p.numel() for p in shared_net.parameters())

print("="*60)
print("VARIANTE A: SharedDQN")
print("="*60)
print(f"\nParámetros totales: {n_params_shared:,}")
print(f"Comparado con independientes: {n_params * 4:,}")
print(f"Reducción: {(1 - n_params_shared / (n_params * 4)) * 100:.1f}%")

```

```

=====
VARIANTE A: SharedDQN
=====

```

```

Parámetros totales: 22,052
Comparado con independientes: 74,276

```

Reducción: 70.3%

6.2 Variante B: CompetitiveDQN (Recompensa por Ranking)

6.2.1 Idea

Modificamos la **función de recompensa** para incentivar competencia: - Bonus por **adelantar** otros coches - Penalización por **ser adelantado**

6.2.2 Ventajas

- Genera comportamientos más agresivos
- Mayor diversidad de estrategias

6.2.3 Desventajas

- Puede generar colisiones intencionales
- Más difícil de converger

```
[9]: def competitive_reward(car, all_cars, old_distance):  
    """  
    Calcula recompensa competitiva basada en ranking.  
  
    Args:  
        car: Coche actual  
        all_cars: Lista de todos los coches  
        old_distance: Distancia del coche antes del paso  
  
    Returns:  
        Recompensa modificada  
    """  
    if not car.alive:  
        return -10.0  
  
    # Recompensa base (igual que original)  
    base_reward = (car.distance - old_distance) * 0.1  
    if car.speed > 0:  
        base_reward += 0.1  
  
    # Calcular ranking (1 = primero, n = último)  
    alive_cars = [c for c in all_cars if c.alive]  
    if len(alive_cars) <= 1:  
        return base_reward + 1.0 # Bonus por ser el último superviviente  
  
    rank = sum(1 for c in alive_cars if c.distance > car.distance) + 1  
    n_alive = len(alive_cars)  
  
    # Bonus/penalización por ranking (0.5 por cada posición)  
    ranking_reward = (n_alive - rank) * 0.2 # Mejor ranking = más recompensa
```

```

        return base_reward + ranking_reward

class CompetitiveRacingGame(RacingGame):
    """
    Variante del juego con recompensa competitiva.
    """

    def step(self, actions):
        """Paso con recompensa competitiva."""
        states = []
        rewards = []
        dones = []

        # Guardar distancias anteriores
        old_distances = [car.distance for car in self.cars]

        # Actualizar coches
        for car, action in zip(self.cars, actions):
            car.update(action, self.track)

        # Calcular recompensas competitivas
        for i, car in enumerate(self.cars):
            reward = competitive_reward(car, self.cars, old_distances[i])
            states.append(car.get_state())
            rewards.append(reward)
            dones.append(not car.alive)

        all_done = all(not car.alive for car in self.cars)
        return states, rewards, dones, all_done

print("="*60)
print("VARIANTE B: CompetitiveDQN")
print("="*60)
print("\nEstructura de recompensa:")
print("  Base: +0.1 × Δdistancia + 0.1 (si v > 0)")
print("  Ranking: +0.2 × (n_alive - rank)")
print("  Colisión: -10")
print("  Último superviviente: +1.0 bonus")

```

```

=====
VARIANTE B: CompetitiveDQN
=====

```

Estructura de recompensa:

Base: +0.1 × Δdistancia + 0.1 (si v > 0)

Ranking: $+0.2 \times (n_alive - rank)$
Colisión: -10
Último superviviente: +1.0 bonus

6.3 Variante C: CooperativeDQN (Recompensa Grupal)

6.3.1 Idea

En lugar de competir, los coches **cooperan**: - Bonus si **todos avanzan** - Penalización si chocan entre ellos

6.3.2 Ventajas

- Comportamientos más seguros
- Los coches aprenden a evitarse

6.3.3 Desventajas

- Puede ser menos “emocionante”
- Más difícil definir la recompensa

```
[10]: def cooperative_reward(car, all_cars, old_distance):  
    """  
    Calcula recompensa cooperativa basada en rendimiento grupal.  
  
    Args:  
        car: Coche actual  
        all_cars: Lista de todos los coches  
        old_distance: Distancia del coche antes del paso  
  
    Returns:  
        Recompensa modificada  
    """  
    if not car.alive:  
        return -10.0  
  
    # Recompensa base individual  
    individual_reward = (car.distance - old_distance) * 0.1  
    if car.speed > 0:  
        individual_reward += 0.05 # Reducido para dar más peso al grupo  
  
    # Recompensa grupal  
    alive_cars = [c for c in all_cars if c.alive]  
    n_alive = len(alive_cars)  
  
    # Bonus por supervivencia grupal  
    survival_bonus = n_alive * 0.05 # Más coches vivos = mejor  
  
    # Bonus por distancia promedio del grupo
```

```

    if n_alive > 1:
        avg_distance = np.mean([c.distance for c in alive_cars])
        group_progress = avg_distance * 0.001 # Pequeño bonus por progreso
    else:
        group_progress = 0

    return individual_reward + survival_bonus + group_progress

class CooperativeRacingGame(RacingGame):
    """
    Variante del juego con recompensa cooperativa.
    """

    def step(self, actions):
        """Paso con recompensa cooperativa."""
        states = []
        rewards = []
        dones = []

        old_distances = [car.distance for car in self.cars]

        for car, action in zip(self.cars, actions):
            car.update(action, self.track)

        for i, car in enumerate(self.cars):
            reward = cooperative_reward(car, self.cars, old_distances[i])
            states.append(car.get_state())
            rewards.append(reward)
            dones.append(not car.alive)

        all_done = all(not car.alive for car in self.cars)
        return states, rewards, dones, all_done

print("="*60)
print("VARIANTE C: CooperativeDQN")
print("="*60)
print("\nEstructura de recompensa:")
print("  Individual: +0.1 × Δdistancia + 0.05 (si v > 0)")
print("  Supervivencia: +0.05 × n_alive")
print("  Progreso grupal: +0.001 × avg_distance")
print("  Colisión: -10")

```

```

=====
VARIANTE C: CooperativeDQN
=====

```

Estructura de recompensa:

Individual: $+0.1 \times \Delta \text{distancia} + 0.05$ (si $v > 0$)
Supervivencia: $+0.05 \times n_alive$
Progreso grupal: $+0.001 \times \text{avg_distance}$
Colisión: -10

6.4 Variante D: A2C Multi-Agente (Policy Gradient)

6.4.1 Idea

Usar **Advantage Actor-Critic (A2C)** en lugar de DQN: - **Actor**: Política que elige acciones directamente - **Critic**: Estima el valor del estado

6.4.2 Ventajas

- Más estable para algunos problemas
- Funciona bien con acciones continuas

6.4.3 Desventajas

- Menos eficiente en datos (on-policy)
- Más hiperparámetros

```
[11]: class ActorCritic(nn.Module):
    """
    Red Actor-Critic para un coche.

    Actor: Produce distribución de probabilidad sobre acciones
    Critic: Estima  $V(s)$ 
    """

    def __init__(self, state_size=6, n_actions=9):
        super().__init__()

        # Backbone compartido
        self.shared = nn.Sequential(
            nn.Linear(state_size, 128),
            nn.ReLU(),
            nn.Linear(128, 128),
            nn.ReLU()
        )

        # Actor head (política)
        self.actor = nn.Sequential(
            nn.Linear(128, n_actions),
            nn.Softmax(dim=-1)
        )
```

```

        # Critic head (valor)
        self.critic = nn.Linear(128, 1)

    def forward(self, x):
        shared = self.shared(x)
        policy = self.actor(shared) # Probabilidades de acciones
        value = self.critic(shared) # V(s)
        return policy, value

class A2CAgent:
    """
    Agente A2C para un coche individual.
    """

    def __init__(self, agent_id, state_size=6, n_actions=9):
        self.id = agent_id
        self.n_actions = n_actions
        self.gamma = 0.95
        self.entropy_coef = 0.01 # Para exploración

        self.device = torch.device("cuda" if torch.cuda.is_available() else ↪
        ↪ "cpu")

        self.network = ActorCritic(state_size, n_actions).to(self.device)
        self.optimizer = optim.Adam(self.network.parameters(), lr=0.001)

        # Almacenar trayectoria del episodio
        self.states = []
        self.actions = []
        self.rewards = []
        self.values = []
        self.log_probs = []

    def act(self, state):
        state_t = torch.FloatTensor(state).unsqueeze(0).to(self.device)

        with torch.no_grad():
            policy, value = self.network(state_t)

        # Muestrear acción de la distribución
        dist = torch.distributions.Categorical(policy)
        action = dist.sample()

        # Guardar para entrenamiento
        self.states.append(state)
        self.actions.append(action.item())

```

```

        self.values.append(value.item())
        self.log_probs.append(dist.log_prob(action).item())

    return action.item()

def remember_reward(self, reward):
    self.rewards.append(reward)

def learn(self):
    """Entrena al final del episodio."""
    if len(self.rewards) == 0:
        return 0

    # Calcular returns y advantages
    returns = []
    R = 0
    for r in reversed(self.rewards):
        R = r + self.gamma * R
        returns.insert(0, R)

    returns = torch.FloatTensor(returns).to(self.device)
    values = torch.FloatTensor(self.values).to(self.device)
    log_probs = torch.FloatTensor(self.log_probs).to(self.device)

    # Normalizar returns
    returns = (returns - returns.mean()) / (returns.std() + 1e-8)

    # Advantage = Returns - Values
    advantages = returns - values

    # Actor loss (policy gradient)
    actor_loss = -(log_probs * advantages.detach()).mean()

    # Critic loss (value function)
    critic_loss = nn.MSELoss()(values, returns)

    # Entropy para exploración
    states_t = torch.FloatTensor(np.array(self.states)).to(self.device)
    policy, _ = self.network(states_t)
    entropy = -(policy * torch.log(policy + 1e-8)).sum(dim=-1).mean()

    # Loss total
    loss = actor_loss + 0.5 * critic_loss - self.entropy_coef * entropy

    self.optimizer.zero_grad()
    loss.backward()
    self.optimizer.step()

```

```

        # Limpiar buffers
        self.states = []
        self.actions = []
        self.rewards = []
        self.values = []
        self.log_probs = []

        return loss.item()

# Verificar arquitectura
ac_net = ActorCritic(state_size=6, n_actions=9)
n_params_ac = sum(p.numel() for p in ac_net.parameters())

print("="*60)
print("VARIANTE D: A2C Multi-Agente")
print("="*60)
print(f"\n{ac_net}")
print(f"\nParámetros por agente: {n_params_ac:,}")
print(f"Con 4 coches: {n_params_ac * 4:,}")

```

```

=====
VARIANTE D: A2C Multi-Agente
=====

```

```

ActorCritic(
  (shared): Sequential(
    (0): Linear(in_features=6, out_features=128, bias=True)
    (1): ReLU()
    (2): Linear(in_features=128, out_features=128, bias=True)
    (3): ReLU()
  )
  (actor): Sequential(
    (0): Linear(in_features=128, out_features=9, bias=True)
    (1): Softmax(dim=-1)
  )
  (critic): Linear(in_features=128, out_features=1, bias=True)
)

```

```

Parámetros por agente: 18,698
Con 4 coches: 74,792

```

6.5 Variante E: EvolutionaryDQN (Algoritmo Evolutivo)

6.5.1 Idea

En lugar de backpropagation, usamos **evolución**: - Torneo entre agentes - Los mejores “sobreviven”
- Mutación de pesos

6.5.2 Ventajas

- Explora mejor el espacio de soluciones
- No requiere gradientes

6.5.3 Desventajas

- Convergencia más lenta
- Menos eficiente en datos

```
[12]: class EvolutionaryAgent:
    """
    Agente que evoluciona mediante selección y mutación.
    """

    def __init__(self, agent_id, state_size=6, n_actions=9):
        self.id = agent_id
        self.n_actions = n_actions

        self.device = torch.device("cuda" if torch.cuda.is_available() else ↪
"cpu")

        # Red simple (sin target network ni replay buffer)
        self.network = CarDQN(state_size, n_actions).to(self.device)
        self.fitness = 0

    def act(self, state):
        """Acción greedy (sin exploración )."""
        with torch.no_grad():
            state_t = torch.FloatTensor(state).unsqueeze(0).to(self.device)
            q_values = self.network(state_t)
            return q_values.argmax().item()

    def mutate(self, mutation_rate=0.1, mutation_strength=0.1):
        """
        Muta los pesos de la red.

        Args:
            mutation_rate: Probabilidad de mutar cada peso
            mutation_strength: Magnitud de la mutación
        """
        with torch.no_grad():
```

```

        for param in self.network.parameters():
            mask = torch.rand_like(param) < mutation_rate
            noise = torch.randn_like(param) * mutation_strength
            param.add_(mask.float() * noise)

    def copy_from(self, other):
        """Copia los pesos de otro agente."""
        self.network.load_state_dict(other.network.state_dict())

def evolve_population(agents, fitness_scores, elite_ratio=0.25, mutation_rate=0.
↳1):
    """
    Evoluciona la población de agentes.

    Args:
    agents: Lista de agentes
    fitness_scores: Lista de fitness de cada agente
    elite_ratio: Porcentaje de élite que sobrevive
    mutation_rate: Tasa de mutación
    """

    n = len(agents)
    n_elite = max(1, int(n * elite_ratio))

    # Ordenar por fitness (mejores primero)
    sorted_indices = np.argsort(fitness_scores[::-1])

    # Élite: los mejores sobreviven sin cambios
    elite_indices = sorted_indices[:n_elite]

    # El resto: copiar de élite y mutar
    for i in sorted_indices[n_elite:]:
        # Elegir padre de la élite
        parent_idx = elite_indices[random.randint(0, n_elite - 1)]
        agents[i].copy_from(agents[parent_idx])
        agents[i].mutate(mutation_rate)

    return agents

print("="*60)
print("VARIANTE E: EvolutionaryDQN")
print("="*60)
print("\nProceso evolutivo:")
print("  1. Evaluar fitness de cada agente")
print("  2. Seleccionar élite (top 25%)")
print("  3. Clonar élite para reemplazar peores")

```

```
print(" 4. Mutar clones (10% de pesos, =0.1)")
print(" 5. Repetir")
```

```
=====
VARIANTE E: EvolutionaryDQN
=====
```

Proceso evolutivo:

1. Evaluar fitness de cada agente
2. Seleccionar elite (top 25%)
3. Clonar elite para reemplazar peores
4. Mutar clones (10% de pesos, =0.1)
5. Repetir

7 6. Comparación de Variantes

Entrenamos todas las variantes y comparamos:

```
[13]: def entrenar_variante_shared(n_cars=4, episodes=30, max_steps=500):
    """Entrena con SharedDQN."""
    print("\nEntrenando SharedDQN...")

    game = RacingGame(n_cars=n_cars, render=False)
    agent = SharedCarAgent(n_cars=n_cars)

    fitness_history = []

    for ep in range(episodes):
        states = game.reset()

        for step in range(max_steps):
            actions = [agent.act(states[i], i) for i in range(n_cars)]
            next_states, rewards, dones, all_done = game.step(actions)

            for i in range(n_cars):
                agent.remember(i, states[i], actions[i], rewards[i],
                               next_states[i], dones[i])

            agent.replay()
            states = next_states

            if all_done:
                break

        agent.decay_epsilon()
        if ep % 10 == 0:
```

```

        agent.update_target()

    avg_fitness = np.mean([car.fitness for car in game.cars])
    fitness_history.append(avg_fitness)

    if (ep + 1) % 10 == 0:
        print(f"  Ep {ep+1}: Fitness = {avg_fitness:.0f}")

    return fitness_history

def entrenar_variante_competitive(n_cars=4, episodes=30, max_steps=500):
    """Entrena con recompensa competitiva."""
    print("\nEntrenando CompetitiveDQN...")

    game = CompetitiveRacingGame(n_cars=n_cars, render=False)
    agents = [CarAgent(i) for i in range(n_cars)]

    fitness_history = []

    for ep in range(episodes):
        states = game.reset()

        for step in range(max_steps):
            actions = [agent.act(state) for agent, state in zip(agents, states)]
            next_states, rewards, done, all_done = game.step(actions)

            for i, agent in enumerate(agents):
                agent.remember(states[i], actions[i], rewards[i],
                                next_states[i], done[i])
                agent.replay()

            states = next_states

            if all_done:
                break

        for agent in agents:
            agent.decay_epsilon()
            if ep % 10 == 0:
                agent.update_target()

        avg_fitness = np.mean([car.fitness for car in game.cars])
        fitness_history.append(avg_fitness)

        if (ep + 1) % 10 == 0:
            print(f"  Ep {ep+1}: Fitness = {avg_fitness:.0f}")

```

```

    return fitness_history

def entrenar_variante_cooperativo(n_cars=4, episodes=30, max_steps=500):
    """Entrena con recompensa cooperativa."""
    print("\nEntrenando CooperativeDQN...")

    game = CooperativeRacingGame(n_cars=n_cars, render=False)
    agents = [CarAgent(i) for i in range(n_cars)]

    fitness_history = []

    for ep in range(episodes):
        states = game.reset()

        for step in range(max_steps):
            actions = [agent.act(state) for agent, state in zip(agents, states)]
            next_states, rewards, dones, all_done = game.step(actions)

            for i, agent in enumerate(agents):
                agent.remember(states[i], actions[i], rewards[i],
                                next_states[i], dones[i])
                agent.replay()

            states = next_states

            if all_done:
                break

        for agent in agents:
            agent.decay_epsilon()
            if ep % 10 == 0:
                agent.update_target()

        avg_fitness = np.mean([car.fitness for car in game.cars])
        fitness_history.append(avg_fitness)

        if (ep + 1) % 10 == 0:
            print(f"  Ep {ep+1}: Fitness = {avg_fitness:.0f}")

    return fitness_history

```

```

[14]: # Entrenar todas las variantes
print("="*60)
print("COMPARACIÓN DE VARIANTES")
print("="*60)

```

```

# Original (agentes independientes)
print("\nEntrenando Original (Independientes)...")
_, history_original = entrenar_demo(n_cars=4, episodes=30, render_cada=0)
avg_original = np.mean(history_original, axis=0)

# Shared
history_shared = entrenar_variante_shared(episodes=30)

# Competitive
history_competitive = entrenar_variante_competitive(episodes=30)

# Cooperative
history_cooperative = entrenar_variante_cooperative(episodes=30)

```

COMPARACIÓN DE VARIANTES

Entrenando Original (Independientes)...

ENTRENAMIENTO DEMO - 4 Coches Independientes

```

Ep   5 | Fitness: 172 | : 0.975
Ep  10 | Fitness: 182 | : 0.951
Ep  15 | Fitness: 194 | : 0.928
Ep  20 | Fitness: 169 | : 0.905
Ep  25 | Fitness: 155 | : 0.882
Ep  30 | Fitness: 169 | : 0.860

```

Entrenando SharedDQN...

```

Ep 10: Fitness = 179
Ep 20: Fitness = 253
Ep 30: Fitness = 190

```

Entrenando CompetitiveDQN...

```

Ep 10: Fitness = 196
Ep 20: Fitness = 186
Ep 30: Fitness = 244

```

Entrenando CooperativeDQN...

```

Ep 10: Fitness = 182
Ep 20: Fitness = 197
Ep 30: Fitness = 167

```

```

[15]: # Visualizar comparación
plt.figure(figsize=(14, 5))

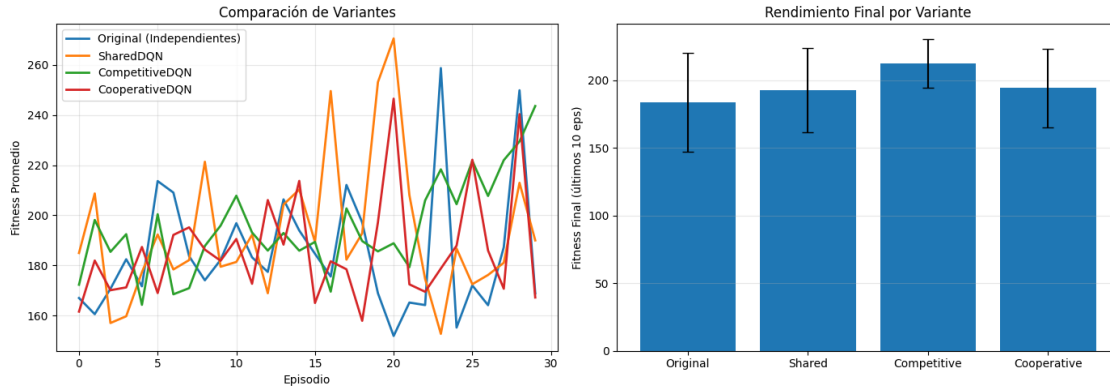
# Curvas de aprendizaje
plt.subplot(1, 2, 1)
plt.plot(avg_original, label='Original (Independientes)', linewidth=2)
plt.plot(history_shared, label='SharedDQN', linewidth=2)
plt.plot(history_competitive, label='CompetitiveDQN', linewidth=2)
plt.plot(history_cooperative, label='CooperativeDQN', linewidth=2)
plt.xlabel('Episodio')
plt.ylabel('Fitness Promedio')
plt.title('Comparación de Variantes')
plt.legend()
plt.grid(True, alpha=0.3)

# Fitness final (boxplot)
plt.subplot(1, 2, 2)
data = [
    avg_original[-10:],
    history_shared[-10:],
    history_competitive[-10:],
    history_cooperative[-10:]
]
labels = ['Original', 'Shared', 'Competitive', 'Cooperative']
plt.bar(labels, [np.mean(d) for d in data], yerr=[np.std(d) for d in data],
        capsize=5)
plt.ylabel('Fitness Final (últimos 10 eps)')
plt.title('Rendimiento Final por Variante')
plt.grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.show()

# Tabla resumen
print("\n" + "="*60)
print("RESUMEN DE RESULTADOS")
print("="*60)
print(f"\n{'Variante':<20} {'Fitness Final':<15} {'Std':<10}")
print("-" * 45)
for name, hist in zip(labels, data):
    print(f"{name:<20} {np.mean(hist):<15.1f} {np.std(hist):<10.1f}")

```



RESUMEN DE RESULTADOS

Variante	Fitness Final	Std
Original	183.7	36.5
Shared	192.5	30.8
Competitive	212.1	18.1
Cooperative	194.1	28.9

8 7. Conclusiones

8.1 ¿Qué aprendimos?

1. Arquitectura Multi-Agente:

- La implementación original usa **agentes independientes** (cada coche tiene su propia red)
- Esto es simple pero usa más parámetros

2. Variantes:

- **SharedDQN**: Más eficiente en parámetros, transfer learning
- **Competitive**: Genera comportamientos más agresivos
- **Cooperative**: Comportamientos más seguros
- **A2C**: Policy gradient alternativo
- **Evolutionary**: Exploración sin gradientes

3. Diseño del Estado:

- Los sensores raycast proporcionan información local suficiente
- 6 dimensiones es compacto pero efectivo

8.2 Siguientes Pasos

- Entrenar por más episodios (200+) para ver convergencia real
- Experimentar con diferentes hiperparámetros
- Probar arquitecturas más complejas (CNNs con visualización)
- Implementar comunicación entre agentes

8.3 Referencias

- [Multi-Agent Reinforcement Learning: A Selective Overview](#)
- [Playing Atari with Deep Reinforcement Learning](#)
- [Asynchronous Methods for Deep Reinforcement Learning \(A3C\)](#)

8.4 Variantes de Entrenamiento Multi-Agente

El proyecto Racing es especialmente rico en variantes porque tiene **4 agentes simultáneos**. La forma en que estos agentes comparten (o no) conocimiento define filosofías muy distintas de aprendizaje multi-agente.

Variante	Nombre	Redes	Buffer	Concepto clave
A	Independientes (<i>actual</i>)	4 redes separadas	4 buffers	Cada agente aprende solo
B	Red Compartida	1 red compartida	1 buffer	Centralizado, 4× más datos
C	Maestro-Alumno	1+N redes	independientes	Transfer learning
D	Competitivo	4 redes separadas	4 buffers	Recompensa relativa

8.4.1 Variante A — Agentes Independientes (*implementación actual*)

```
python racing_game.py --train --variant independent
```

Cada coche tiene su **propia red neuronal** [6→128→128→9] y su propio replay buffer (50K experiencias). Aprenden en paralelo pero sin comunicarse.

Cuándo funciona bien: cuando los agentes tienen objetivos distintos o cuando queremos que cada uno desarrolle un estilo propio.

Limitación: cada agente ve solo 1/4 de la experiencia total disponible. Aprende más lento que si pudiera aprender de todos los coches.

```
[16]: # Variante A: Agentes Independientes
# Cada coche tiene su propia red DQN y su propio buffer
import subprocess
# subprocess.run(["python", "racing_game.py", "--train", "--variant",
↪ "independent", "--episodes", "50"])
```

```
# Para ejecutar directamente en este notebook:
import sys, os
os.chdir(os.path.dirname(os.path.abspath("racing_game.py")))

# Importar funciones del archivo principal
# from racing_game import train_agents
# train_agents(n_cars=4, episodes=50)

print("Variante A: 4 agentes con redes independientes")
print(" - 4 redes DQN: [6 → 128 → 128 → 9]")
print(" - 4 replay buffers (50K exp. cada uno)")
print(" - Cada agente entrena con sus propias experiencias")
print(" - Modelos guardados: car_agent_0.pth, car_agent_1.pth, ...")
```

Variante A: 4 agentes con redes independientes

- 4 redes DQN: [6 → 128 → 128 → 9]
- 4 replay buffers (50K exp. cada uno)
- Cada agente entrena con sus propias experiencias
- Modelos guardados: car_agent_0.pth, car_agent_1.pth, ...

8.4.2 Variante B — Red Compartida (Centralized Training)

```
python racing_game.py --train --variant shared
```

Todos los coches usan **la misma red neuronal** y sus experiencias van al **mismo replay buffer**. Es como tener un único agente que se ejecuta en 4 instancias simultáneas.

Ventaja: el buffer acumula 4× más experiencias por episodio → aprende más rápido.

Limitación: todos aprenden la misma política. No pueden especializarse.

Concepto: *Centralized Training, Decentralized Execution* (CTDE) — uno de los paradigmas fundamentales en Multi-Agent RL.

Estado Coche 0

Estado Coche 1 Red Compartida Acción para cada coche

Estado Coche 2

Estado Coche 3

Todas las exp. → mismo buffer

```
[17]: # Variante B: Red Compartida
# from racing_game import train_shared, TORCH_AVAILABLE

# if TORCH_AVAILABLE:
#     train_shared(n_cars=4, episodes=50)

print("Variante B: Red neuronal compartida")
print(" - 1 red DQN compartida: [6 → 128 → 128 → 9]")
print(" - 1 replay buffer compartido (50K exp.)")
print(" - Todas las experiencias de los 4 coches van al mismo buffer")
```

```

print(" - 4× más datos por episodio → convergencia más rápida")
print(" - Modelo guardado: car_shared.pth")

# Diferencia clave en el bucle de entrenamiento:
codigo_compartido = """
# Variante B: un solo agente para todos
shared_agent = SharedCarAgent(n_agents=4)

# Todas las experiencias al mismo buffer
for i in range(n_cars):
    shared_agent.remember(states[i], actions[i], rewards[i], next_states[i],
    done[i])

# Una sola actualización de red
shared_agent.replay() # Entrena con batch del buffer compartido
"""
print("\nCódigo clave (Variante B vs A):")
print(codigo_compartido)

```

Variante B: Red neuronal compartida

- 1 red DQN compartida: [6 → 128 → 128 → 9]
- 1 replay buffer compartido (50K exp.)
- Todas las experiencias de los 4 coches van al mismo buffer
- 4× más datos por episodio → convergencia más rápida
- Modelo guardado: car_shared.pth

Código clave (Variante B vs A):

```

# Variante B: un solo agente para todos
shared_agent = SharedCarAgent(n_agents=4)

# Todas las experiencias al mismo buffer
for i in range(n_cars):
    shared_agent.remember(states[i], actions[i], rewards[i], next_states[i],
    done[i])

# Una sola actualización de red
shared_agent.replay() # Entrena con batch del buffer compartido

```

8.4.3 Variante C — Maestro-Alumno (Transfer Learning)

`python racing_game.py --train --variant master_student`

Fase 1: Un agente *maestro* entrena solo durante N episodios hasta adquirir una política básica.

Fase 2: Los demás agentes *copian* exactamente los pesos del maestro y refinan a partir de ese punto, con epsilon reducido (ya saben algo).

Por qué funciona: el conocimiento de “cómo conducir” es transferible. Los alumnos no empiezan desde exploración aleatoria sino desde una política ya competente.

Analogía: como aprender a conducir viendo primero cómo lo hace un instructor y luego practicando por tu cuenta.

	Sin transfer (Var. A)	Con transfer (Var. C)
Inicio alumnos	= 1.0 (aleatorio)	= 0.3 (ya saben algo)
Episodios útiles desde	~50 ep.	~5 ep.

```
[18]: # Variante C: Maestro-Alumno
# from racing_game import train_master_student

# train_master_student(n_cars=4, master_episodes=50, student_episodes=50)

print("Variante C: Maestro-Alumno")
print("\nFase 1 - El maestro aprende solo:")
print(" - 1 coche, entrena master_episodes episodios")
print(" - Guarda pesos en: car_master.pth")
print("\nFase 2 - Los alumnos copian al maestro:")
print(" - 4 coches cargan car_master.pth")
print(" - epsilon = 0.3 (exploración reducida, ya saben conducir)")
print(" - Refinan durante student_episodes episodios")
print(" - Modelos guardados: car_student_0.pth, car_student_1.pth, ...")

transferencia = """
# Copia de pesos del maestro a los alumnos
for i in range(n_cars):
    student = CarAgent(i)
    student.q_network.load_state_dict(master.q_network.state_dict())
    student.target_network.load_state_dict(master.q_network.state_dict())
    student.epsilon = 0.3 # Reducido: ya saben algo
"""
print("\nCódigo de transferencia de pesos:")
print(transferencia)
```

Variante C: Maestro-Alumno

Fase 1 - El maestro aprende solo:

- 1 coche, entrena master_episodes episodios
- Guarda pesos en: car_master.pth

Fase 2 - Los alumnos copian al maestro:

- 4 coches cargan car_master.pth
- epsilon = 0.3 (exploración reducida, ya saben conducir)
- Refinan durante student_episodes episodios
- Modelos guardados: car_student_0.pth, car_student_1.pth, ...

Código de transferencia de pesos:

```
# Copia de pesos del maestro a los alumnos
for i in range(n_cars):
    student = CarAgent(i)
    student.q_network.load_state_dict(master.q_network.state_dict())
    student.target_network.load_state_dict(master.q_network.state_dict())
    student.epsilon = 0.3 # Reducido: ya saben algo
```

8.4.4 Variante D — Competitivo con Ranking

```
python racing_game.py --train --variant competitive
```

La recompensa ya no es solo por **sobrevivir y avanzar**, sino también por la **posición relativa** respecto al resto de coches vivos.

Cálculo del ranking bonus:

```
rank_bonus    [-0.5, +0.5]
Líder (rank 0):    +0.5
Último (rank n-1): -0.5
```

Recompensa total: `base_reward + rank_bonus`

Esto introduce **competencia explícita**: no basta con mantenerse en pista, hay que hacerlo *mejor que los demás*.

Comportamiento emergente esperado: los agentes aprenden a ser más agresivos, priorizando adelantar en lugar de simplemente sobrevivir.

```
[19]: # Variante D: Competitivo con ranking
      # from racing_game import train_competitive

      # train_competitive(n_cars=4, episodes=50)

      print("Variante D: Competitivo con Ranking")
      print(" - 4 redes DQN independientes (igual que Var. A)")
      print(" - Recompensa base: avanzar + sobrevivir")
      print(" - Bonus adicional por posición relativa: [-0.5, +0.5]")

      ranking_code = """
      # Calcular ranking por fitness (mayor fitness = mejor posición)
      alive_indices = [i for i, car in enumerate(game.cars) if car.alive]
      sorted_alive = sorted(alive_indices, key=lambda i: game.cars[i].fitness,
                           ↪reverse=True)

      for rank, idx in enumerate(sorted_alive):
          rank_bonus = 0.5 * (1 - 2 * rank / max(n_alive - 1, 1))
          competitive_rewards[idx] += rank_bonus
```

```

    # rank 0 (líder) → +0.5
    # rank 3 (último) → -0.5
"""
print("\nCódigo de recompensa competitiva:")
print(ranking_code)

print("\nComparación de filosofías:")
print(" Variante A/B: 'Sobrevive y avanza' (cooperativo implícito)")
print(" Variante D: 'Sé mejor que los demás' (competitivo explícito)")

```

Variante D: Competitivo con Ranking

- 4 redes DQN independientes (igual que Var. A)
- Recompensa base: avanzar + sobrevivir
- Bonus adicional por posición relativa: [-0.5, +0.5]

Código de recompensa competitiva:

```

# Calcular ranking por fitness (mayor fitness = mejor posición)
alive_indices = [i for i, car in enumerate(game.cars) if car.alive]
sorted_alive = sorted(alive_indices, key=lambda i: game.cars[i].fitness,
reverse=True)

for rank, idx in enumerate(sorted_alive):
    rank_bonus = 0.5 * (1 - 2 * rank / max(n_alive - 1, 1))
    competitive_rewards[idx] += rank_bonus
    # rank 0 (líder) → +0.5
    # rank 3 (último) → -0.5

```

Comparación de filosofías:

Variante A/B: 'Sobrevive y avanza' (cooperativo implícito)
 Variante D: 'Sé mejor que los demás' (competitivo explícito)

8.4.5 Comparativa de Variantes

Aspecto	A: Independ.	B: Compartida	C: Maestro	D: Competitivo
Convergencia	Media	Más rápida	Rápida inicial	Media
Especialización	Sí	No	Parcial	Sí
Datos por ep.	1×	4×	1× (fase 2)	1×
Complejidad impl.	Baja	Baja	Media	Media
Comportamiento	Neutral	Homogéneo	Variado	Agresivo

¿Cuándo usar cada una? - **A** — Baseline, cuando quieres comparar con otras variantes - **B** — Cuando el tiempo de convergencia es prioritario - **C** — Cuando tienes un agente pre-entrenado o quieres demostrar transfer learning - **D** — Cuando quieres comportamiento emergente competitivo